

MAKALAH
“TUGAS 3 SISTEM OPRASI”

Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Oprasi

Dosen : Triawan Adi Cahyanto, M.Kom



Disusun Oleh:
Nama: Ragil Anggara Kasih Budiarto

NIM:2410651045

Prodi:Teknik Informatika

Kelas: TI – C

UNIVERSITAS MUHAMMADIYAH JEMBER

2025-2026

BAB I KALKULATOR PARALEL

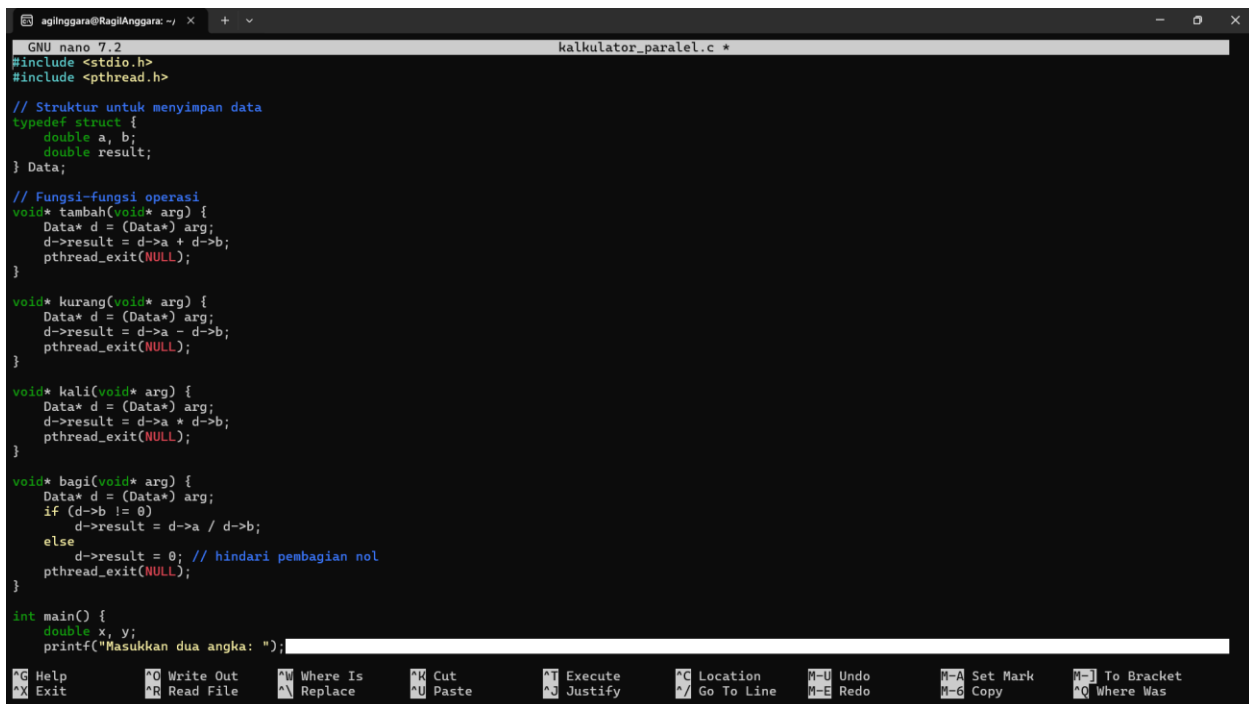
Buat program yang:

1. Menerima input 2 angka dari user
2. Menghitung penjumlahan, pengurangan, perkalian, dan pembagian
3. Setiap operasi dikerjakan oleh thread berbeda
4. Tampilkan semua hasil setelah semua thread selesai

Langkah Langkah:

Buat file kalkulator_paralel.c

```
agilnggara@RagilAnggara:~/tugas3$ mkdir kalkulator_paralel
agilnggara@RagilAnggara:~/tugas3$ cd kalkulator_paralel
agilnggara@RagilAnggara:~/tugas3/kalkulator_paralel$ nano kalkulator_paralel.c
```



```
GNU nano 7.2 kalkulator_paralel.c *
#include <stdio.h>
#include <pthread.h>

// Struktur untuk menyimpan data
typedef struct {
    double a, b;
    double result;
} Data;

// Fungsi-fungsi operasi
void* tambah(void* arg) {
    Data* d = (Data*) arg;
    d->result = d->a + d->b;
    pthread_exit(NULL);
}

void* kurang(void* arg) {
    Data* d = (Data*) arg;
    d->result = d->a - d->b;
    pthread_exit(NULL);
}

void* kali(void* arg) {
    Data* d = (Data*) arg;
    d->result = d->a * d->b;
    pthread_exit(NULL);
}

void* bagi(void* arg) {
    Data* d = (Data*) arg;
    if (d->b != 0)
        d->result = d->a / d->b;
    else
        d->result = 0; // hindari pembagian nol
    pthread_exit(NULL);
}

int main() {
    double x, y;
    printf("Masukkan dua angka: ");
```

```

scanf("%lf %lf", &x, &y);

pthread_t t1, t2, t3, t4;
Data d1 = {x, y, 0}, d2 = {x, y, 0}, d3 = {x, y, 0}, d4 = {x, y, 0};

// Buat thread
pthread_create(&t1, NULL, tambah, &d1);
pthread_create(&t2, NULL, kurang, &d2);
pthread_create(&t3, NULL, kali, &d3);
pthread_create(&t4, NULL, bagi, &d4);

// Tunggu semua thread selesai
pthread_join(t1, NULL);
pthread_join(t2, NULL);
pthread_join(t3, NULL);
pthread_join(t4, NULL);

// Tampilkan hasil
printf("\n== Hasil Perhitungan ==\n");
printf("Penjumlahan : %.2lf\n", d1.result);
printf("Pengurangan : %.2lf\n", d2.result);
printf("Perkalian : %.2lf\n", d3.result);
printf("Pembagian : %.2lf\n", d4.result);

return 0;
}

```

Penjelasan:

- ❖ #include <stdio.h>
 - Ini untuk menjalankan fungsi input output seperti printf, dan scanf
- ❖ #include <pthread.h>
 - Ini adalah library untuk membuat dan juga mengelola thread
- ❖ typedef struct { double a, b, result; } Data;
 - ini adalah struktur untuk menyimpan dua angka dan hasil perhitungan
- ❖ void* tambah(void* arg)
 - Fungsi ini adalah yang dijalankan oleh thread untuk operasi (tambah, kurang, kali, bagi)
- ❖ Data* d = (Data*) arg;
 - Ini untuk mengubah argumen menjadi pointer ke struct data
- ❖ d->result = d->a + d->b;
 - ini digunakan untuk melakukan operasi yang sesuai dan menyimpan hasilnya.
- ❖ pthread_exit(NULL);
 - ini berfungsi untuk mengakhiri thread setelah operasi selesai.
- ❖ pthread_t t1, t2, t3, t4;
 - ini adalah variabel untuk menyimpan ID masing-masing thread.
- ❖ pthread_create(&t1, NULL, tambah, &d1);
 - ini digunakan untuk membuat thread baru untuk menjalankan fungsi tambah
- ❖ pthread_join(t1, NULL);
 - ini berfungsi untuk menunggu thread t1 selesai sebelum lanjut ke thread selanjutnya

❖ printf("...")

- ini memiliki fungsi sebagai menampilkan hasil pekerjaan ke layar

Output:

```
agilinggara@RagilAnggara:~/tugas3/kalkulator_paralel$ gcc kalkulator_paralel.c -o kalkulator_paralel -lpthread
agilinggara@RagilAnggara:~/tugas3/kalkulator_paralel$ ./kalkulator_paralel
Masukkan dua angka: 5 7

=== Hasil Perhitungan ===
Penjumlahan : 12.00
Pengurangan : -2.00
Perkalian   : 35.00
Pembagian   : 0.71
agilinggara@RagilAnggara:~/tugas3/kalkulator_paralel$ |
```

BAB II FILE PROCESING

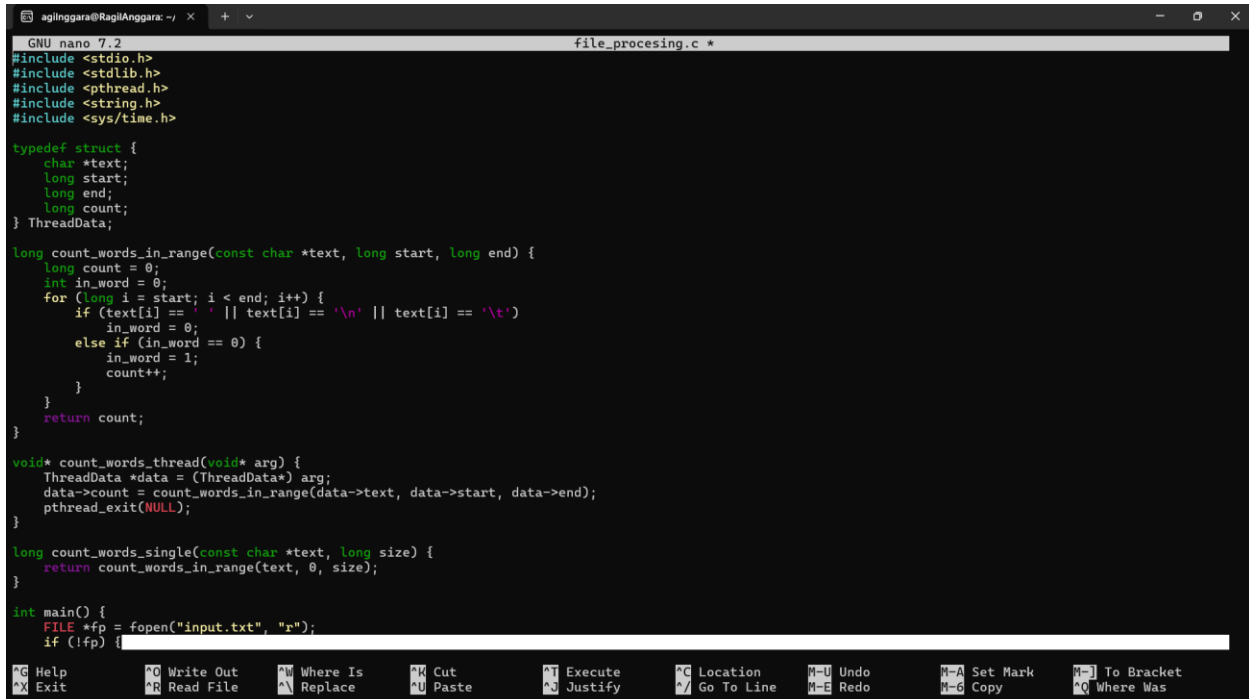
Buat program yang:

1. Membaca file teks
2. Menghitung jumlah kata menggunakan 2 thread (bagi file jadi 2 bagian)
3. Gabungkan hasilnya
4. Bandingkan waktu eksekusi dengan versi single-thread

Langkah Langkah:

Buat file `file_procesing.c`

```
agilnggara@RagilAnggara:~$ cd tugas3
agilnggara@RagilAnggara:~/tugas3$ mkdir file_procesing
agilnggara@RagilAnggara:~/tugas3$ cd file_procesing
agilnggara@RagilAnggara:~/tugas3/file_procesing$ nano file_procesing.c
```



```
GNU nano 7.2 file_procesing.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <sys/time.h>

typedef struct {
    char *text;
    long start;
    long end;
    long count;
} ThreadData;

long count_words_in_range(const char *text, long start, long end) {
    long count = 0;
    int in_word = 0;
    for (long i = start; i < end; i++) {
        if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t')
            in_word = 0;
        else if (in_word == 0) {
            in_word = 1;
            count++;
        }
    }
    return count;
}

void* count_words_thread(void* arg) {
    ThreadData *data = (ThreadData*) arg;
    data->count = count_words_in_range(data->text, data->start, data->end);
    pthread_exit(NULL);
}

long count_words_single(const char *text, long size) {
    return count_words_in_range(text, 0, size);
}

int main() {
    FILE *fp = fopen("input.txt", "r");
    if (!fp) {
```

```
GNU nano 7.2 file_processing.c *
    perror("Gagal membuka file");
    return 1;
}

fseek(fp, 0, SEEK_END);
long size = ftell(fp);
rewind(fp);

char *buffer = (char*) malloc(size + 1);
fread(buffer, 1, size, fp);
buffer[size] = '\0';
fclose(fp);

struct timeval start, end;
double time_single, time_multi;

// --- Single-thread ---
gettimeofday(&start, NULL);
long single_count = count_words_single(buffer, size);
gettimeofday(&end, NULL);
time_single = (end.tv_sec - start.tv_sec) * 1000.0 +
              (end.tv_usec - start.tv_usec) / 1000.0;

// --- Multi-thread ---
ThreadData d1 = {buffer, 0, size / 2, 0};
ThreadData d2 = {buffer, size / 2, size, 0};

pthread_t t1, t2;
gettimeofday(&start, NULL);
pthread_create(&t1, NULL, count_words_thread, &d1);
pthread_create(&t2, NULL, count_words_thread, &d2);
pthread_join(t1, NULL);
pthread_join(t2, NULL);
gettimeofday(&end, NULL);
time_multi = (end.tv_sec - start.tv_sec) * 1000.0 +
             (end.tv_usec - start.tv_usec) / 1000.0;

long total_words = d1.count + d2.count;

printf("=== HASIL PERHITUNGAN ===\n");

printf("Jumlah kata (single-thread) : %ld\n", single_count);
printf("Jumlah kata (multi-thread) : %ld\n", total_words);
printf("\n=== WAKTU EKSEKUSI ===\n");
printf("Single-thread : %.3f ms\n", time_single);
printf("Multi-thread : %.3f ms\n", time_multi);

free(buffer);
return 0;
}
```

Penjelasan:

- ❖ `#include <pthread.h>`
 - Ini berguna untuk membuat dan menjalankan thread
- ❖ `typedef struct { char *text; long start, end, count; } ThreadData;`
 - ini digunakan untuk menyimpan potongan teks dan hasil hitung setiap thread
- ❖ `void* count_words_thread(void* arg)`
 - ini adalah fungsi yang di ajalakan oleh thread untuk menghitung suatu kata
- ❖ `pthread_t t1, t2;`
 - ini adalah variable untuk dua thread
- ❖ `pthread_create(&t1, NULL, count_words_thread, &d1);`
 - ini digunakan nutuk membuat thread pertama
- ❖ `pthread_create(&t2, NULL, count_words_thread, &d2);`
 - ini digunakan untuk membuat thread kedua
- ❖ `pthread_join(t1, NULL); pthread_join(t2, NULL);`

- ini berfungsi untuk menunggu kedua thread selesai
- ❖ `count_words_single(buffer, size);`
 - ini berguna untuk menghitung jumlah kata tanpa thread (Pembandingnya)
- ❖ `printf("Multi-thread: %ld", total_words);`
 - ini berfungsi untuk menampilkan hasil perhitungan multi thread
- ❖ `printf("Single-thread: %ld", single_count);`
 - ini berfungsi untuk menampilkan hasil single thread

Output:

```

agilnggara@RagilAnggara:~/tugas3/file_procesing$ nano input.txt

```



```

GNU nano 7.2 input.txt
nama saya ragil anggara nim 2410651045

agilnggara@RagilAnggara:~/tugas3/file_procesing$ gcc file_procesing.c -o file_procesing -lpthread
agilnggara@RagilAnggara:~/tugas3/file_procesing$ ./file_procesing
=== HASIL PERHITUNGAN ===
Jumlah kata (single-thread) : 6
Jumlah kata (multi-thread) : 7

=== WAKTU EKSEKUSI ===
Single-thread : 0.001 ms
Multi-thread : 1.413 ms

```

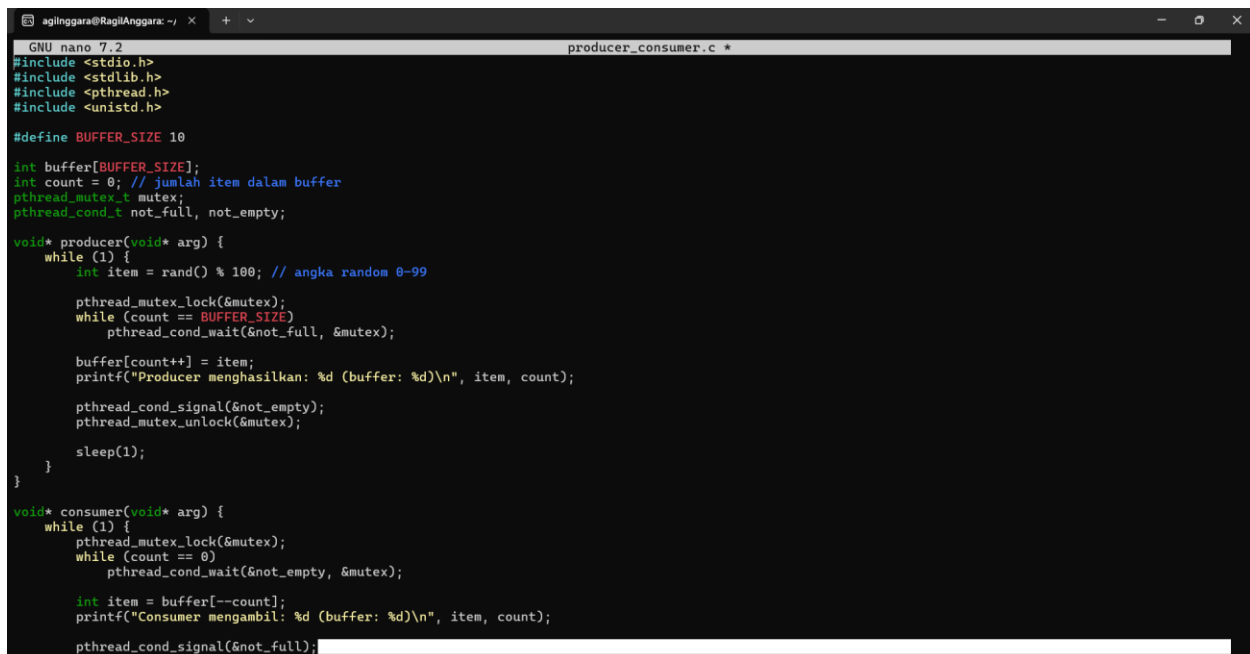
BAB III PRODUCER-CONSUMER

Buat simulasi producer-consumer dengan:

1. Thread producer yang menghasilkan angka random
2. Thread consumer yang memproses angka tersebut
3. Gunakan mutex untuk sinkronisasi
4. Buffer maksimal 10 item

Langkah Langkah:

Buat file producer_consumer.c:



```
GNU nano 7.2 producer_consumer.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int count = 0; // jumlah item dalam buffer
pthread_mutex_t mutex;
pthread_cond_t not_full, not_empty;

void* producer(void* arg) {
    while (1) {
        int item = rand() % 100; // angka random 0-99

        pthread_mutex_lock(&mutex);
        while (count == BUFFER_SIZE)
            pthread_cond_wait(&not_full, &mutex);

        buffer[count++] = item;
        printf("Producer menghasilkan: %d (buffer: %d)\n", item, count);

        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&mutex);

        sleep(1);
    }
}

void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        while (count == 0)
            pthread_cond_wait(&not_empty, &mutex);

        int item = buffer[--count];
        printf("Consumer mengambil: %d (buffer: %d)\n", item, count);
        pthread_cond_signal(&not_full);
    }
}
```



```

        pthread_mutex_unlock(&mutex);
        sleep(2);
    }
}

int main() {
    pthread_t prod, cons;
    srand(time(NULL));

    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&not_full);
    pthread_cond_destroy(&not_empty);
    return 0;
}

```

Penjelasan:

- ❖ `#include <pthread.h>`
 - Ini berfungsi sebagai library untuk thread dan sinkronisasi.
- ❖ `#define BUFFER_SIZE 10`
 - Ini untuk ukuran maksimal buffer (10 item).
- ❖ `pthread_mutex_t mutex;`
 - ini berfungsi untuk mengunci akses ke buffer agar tidak bentrok.
- ❖ `pthread_cond_t not_full, not_empty;`
 - ini adalah kondisi untuk menunggu saat buffer penuh/kosong.
- ❖ `void* producer(void* arg)`
 - ini untuk thread yang menghasilkan angka random dan menambah ke buffer.
- ❖ `void* consumer(void* arg)`
 - ini untuk thread yang mengambil dan memproses angka dari buffer.
- ❖ `pthread_cond_wait()`
 - ini berfungsi untuk thread menunggu sampai kondisi tertentu (buffer kosong/penuh berubah).
- ❖ `pthread_cond_signal()`
 - ini adalah kondisi untuk memberi sinyal bahwa kondisi sudah berubah.
- ❖ `pthread_create(&prod, NULL, producer, NULL);`
 - ini berfungsi untuk membuat thread producer.
- ❖ `pthread_create(&cons, NULL, consumer, NULL);`
 - ini berfungsi untuk membuat thread consumer.

Output:

```
agilnggara@RagilAnggara:~/tugas3/producer_consumer$ gcc producer_consumer.c -o producer_consumer -lpthread
agilnggara@RagilAnggara:~/tugas3/producer_consumer$ ./producer_consumer
Producer menghasilkan: 43 (buffer: 1)
Consumer mengambil: 43 (buffer: 0)
Producer menghasilkan: 59 (buffer: 1)
Consumer mengambil: 59 (buffer: 0)
Producer menghasilkan: 38 (buffer: 1)
Producer menghasilkan: 94 (buffer: 2)
Consumer mengambil: 94 (buffer: 1)
Producer menghasilkan: 35 (buffer: 2)
Producer menghasilkan: 57 (buffer: 3)
Consumer mengambil: 57 (buffer: 2)
Producer menghasilkan: 70 (buffer: 3)
Producer menghasilkan: 51 (buffer: 4)
Consumer mengambil: 51 (buffer: 3)
Producer menghasilkan: 26 (buffer: 4)
Producer menghasilkan: 23 (buffer: 5)
Producer menghasilkan: 46 (buffer: 6)
Consumer mengambil: 46 (buffer: 5)
Producer menghasilkan: 4 (buffer: 6)
Consumer mengambil: 4 (buffer: 5)
Producer menghasilkan: 85 (buffer: 6)
Producer menghasilkan: 66 (buffer: 7)
Producer menghasilkan: 26 (buffer: 8)
Consumer mengambil: 26 (buffer: 7)
Producer menghasilkan: 94 (buffer: 8)
Producer menghasilkan: 92 (buffer: 9)
Consumer mengambil: 92 (buffer: 8)
Producer menghasilkan: 66 (buffer: 9)
Producer menghasilkan: 10 (buffer: 10)
Consumer mengambil: 10 (buffer: 9)
Producer menghasilkan: 86 (buffer: 10)
Consumer mengambil: 86 (buffer: 9)
Producer menghasilkan: 34 (buffer: 10)
^C
agilnggara@RagilAnggara:~/tugas3/producer_consumer$ |
```